Exp. No.: 5	Experiments on Packet Capture Tool: Wireshark

Aim

To understand the features of Wireshark as a packet capture tool, and to understand encapsulation of information at various layers of the protocol stack by capturing, filtering, and analysing network traffic.

Required Software / Tools

- Wireshark (network protocol analyser)

Theory / Concepts

Packet Sniffer

- Sniffs messages being sent/received from/by the computer.
- Stores and displays the contents of the various protocol fields in the messages.
- Is a passive program:
  - Never sends packets itself.
  - Has no packets addressed to it.
  - Receives a copy of all packets (sent/received).

Packet Sniffer Diagnostic Tools

- tcpdump — e.g., tcpdump -enx host 10.129.41.2 -w exe3.out
- Wireshark — e.g., wireshark -r exe3.out

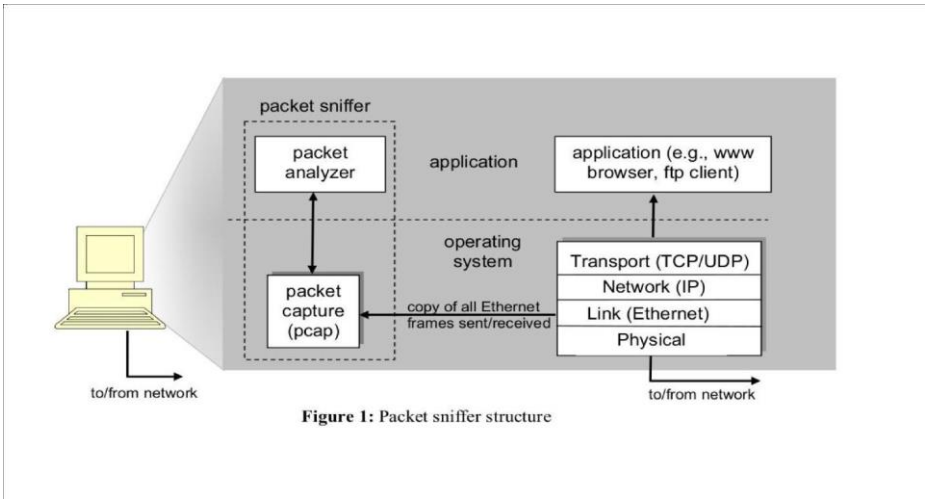


Figure 1: Packet sniffer structure

Fig. 1 — Packet sniffer structure: the packet capture (pcap) layer receives a copy of Ethernet frames sent/received by the protocol stack, and the packet analyser displays them

Wireshark

Wireshark, a network analysis tool formerly known as Ethereal, captures packets in real time and displays them in human-readable format. Wireshark includes filters, colour coding, and other features that let you dig deep into network traffic and inspect individual packets. It can be used to inspect a suspicious program's network traffic, analyse the traffic flow on a network, or troubleshoot network problems.

What Wireshark can do:

- Capture network traffic.
- Decode packet protocols using dissectors.
- Define filters for capture and display.

- Watch smart statistics.
- Analyse problems and interactively browse the traffic.

Wireshark is used by:

- Network administrators — to troubleshoot network problems.
- Network security engineers — to examine security problems.
- Developers — to debug protocol implementations.
- Students — to learn network protocol internals.

Wireshark Interface — Key Panes

After capturing packets, the Wireshark window is divided into three panes:

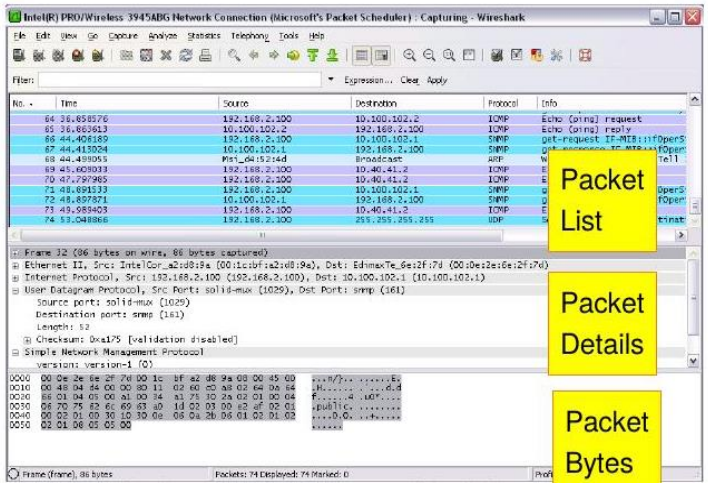


Fig. 2 — Wireshark's Packet List, Packet Details and Packet Bytes panes

Packet List pane:

Displays all the packets in the current capture file. Each line corresponds to one packet; selecting a line displays more details in the Packet Details and Packet Bytes panes.

Packet Details pane:

Shows the currently selected packet in a more detailed, expandable/collapsible tree of protocols and protocol fields.

Packet Bytes pane:

Shows the data of the currently selected packet in hexdump style.

Color Coding:

Wireshark uses colours to help identify traffic types at a glance. By default, light purple is TCP traffic, light blue is UDP traffic, and black identifies packets with errors (e.g., delivered out of order). The colour rules can be viewed and customised via View → Coloring Rules.

Procedure

General steps for capturing and filtering packets:

1. Download and install Wireshark (available for Windows, macOS, and Linux/Unix package repositories).
2. Launch Wireshark and double-click the name of a network interface under Capture to start capturing packets on that interface.
3. Observe packets appearing in real time as they are sent/received by the system. If promiscuous mode is enabled (default), all other packets on the network are also seen — this can be verified under Capture → Options.
4. Click the red Stop button to stop capturing traffic when done.

5. Apply a display filter by typing it into the filter box (e.g., "dns", "arp", "tcp", "http") and pressing Enter, or choose one from Analyze → Display Filters.

6. Right-click a packet and use Follow → TCP Stream to view the full conversation between client and server for that stream.

7. Use Statistics → Flow Graph for a graphical view of the packet exchange sequence.

Lab Exercise: Capturing and Analysing Packets

1. Capture 100 packets from the Ethernet (IEEE 802.3 LAN) interface and save it.

- Select Local Area Connection in Wireshark.
- Go to Capture → Options.
- Select "stop capture automatically after 100 packets".
- Click Start capture, then save the packets.

2. Create a filter to display only TCP/UDP packets, inspect the packets, and provide the flow graph.

- Repeat the capture (100 packets) as above.
- Search TCP packets in the filter/search bar.
- View the flow graph via Statistics → Flow Graph, then save the packets.

3. Create a filter to display only ARP packets and inspect them.

- Repeat the capture (100 packets) as above.
- Search ARP packets in the filter/search bar, then save the packets.

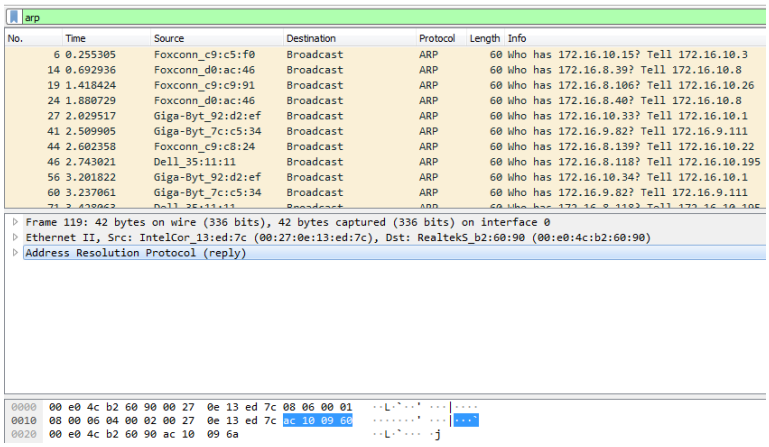


Fig. 3 — Filtered ARP packets showing broadcast "Who has ..." requests

4. Create a filter to display only DNS packets and provide the flow graph.

- Repeat the capture (100 packets) as above.
- Search DNS packets in the filter/search bar.
- View the flow graph via Statistics → Flow Graph, then save the packets.

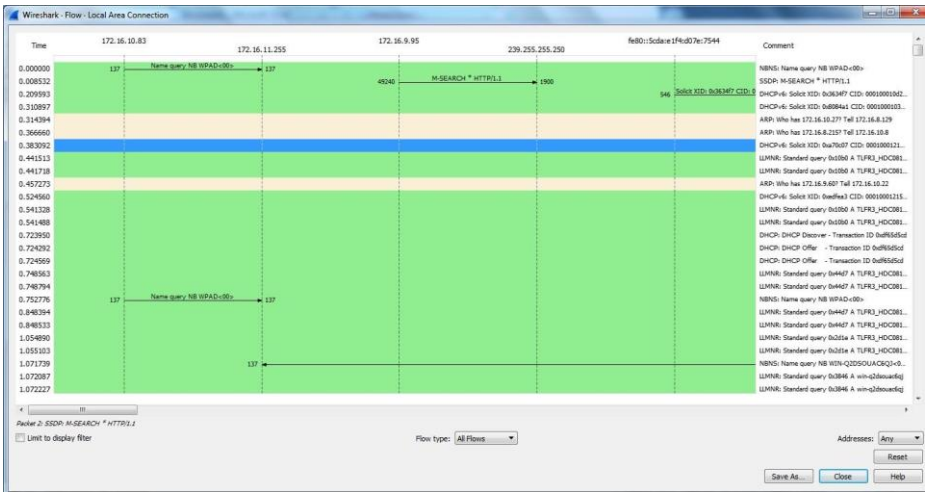


Fig. 4 — Flow graph showing the time sequence of packet exchanges between hosts

5. Create a filter to display only HTTP packets and inspect them.

- Select Local Area Connection in Wireshark and repeat the capture as above.
- Search HTTP packets in the filter/search bar, then save the packets.

6. Create a filter to display only IP/ICMP packets and inspect them.

- Repeat the capture (100 packets) as above.
- Search ICMP/IP packets in the filter/search bar, then save the packets.

7. Create a filter to display only DHCP packets and inspect them.

- Repeat the capture (100 packets) as above.
- Search DHCP packets in the filter/search bar, then save the packets.

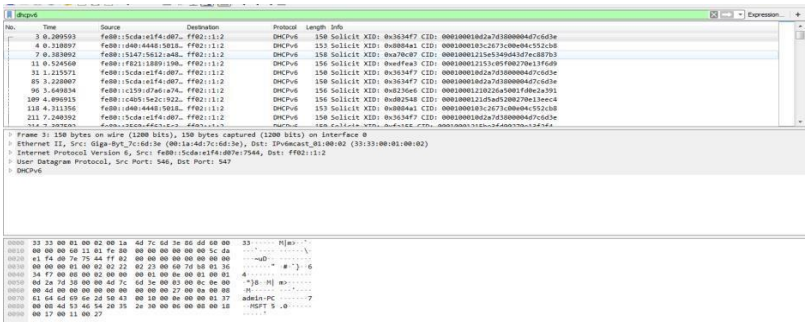


Fig. 5 — Filtered DHCPv6 packets showing Solicit messages

Observation

1. What is promiscuous mode?
2. Does an ARP packet have a transport layer header? Explain.
3. Which transport layer protocol is used by DNS?
4. What is the port number used by the HTTP protocol?
5. What is a broadcast IP address?

Result

Wireshark was used to capture live network traffic, and separate display filters were applied to isolate TCP/UDP, ARP, DNS, HTTP, ICMP/IP, and DHCP packets. The captured packets were inspected at the frame, protocol, and byte level, and flow graphs were generated to visualise the sequence of packet exchanges, thereby demonstrating encapsulation of information across the protocol stack.